Student Management Backend

Google OAuth Testing Using Thunder Client

Browser-based Google authentication + JWT testing in Thunder Client

1. Testing Flow Overview

With the current backend architecture, Google OAuth login should be tested in two parts:

- Browser: starts Google authentication, displays the Google login/consent screen, and receives the OAuth callback.
- Thunder Client: uses the JWT issued by your backend to test protected endpoints such as /api/auth/me and /api/students.

```
Browser
  ↓
GET /api/auth/google
  ↓
Google Login / Consent
  ↓
GET /api/auth/google/callback
  ↓
Backend creates application JWT
  ↓
Thunder Client uses JWT
  ↓
Protected API + RBAC testing
```

2. Verify the .env Configuration

Before testing OAuth, confirm that the backend configuration contains the Google client credentials, JWT configuration, MongoDB URI, and callback URL.

```
PORT=3000
MONGO_URI=mongodb://127.0.0.1:27017/student_management
JWT_SECRET=my_long_secret
JWT_EXPIRES_IN=15m
GOOGLE_CLIENT_ID=xxxxxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=xxxxxxxx
GOOGLE_CALLBACK_URL=http://localhost:3000/api/auth/google/callback
NODE_ENV=development
```

3. Start the Backend

Run the application from the project directory:

```
npm start

# or
node server.js
```

A successful startup should show that MongoDB is connected and that the Express server is listening on the configured port.

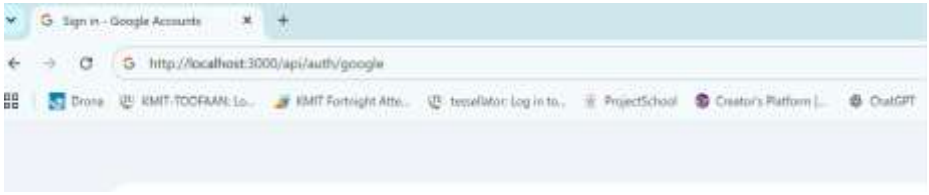
```
MongoDB connected
Server running on 3000
```

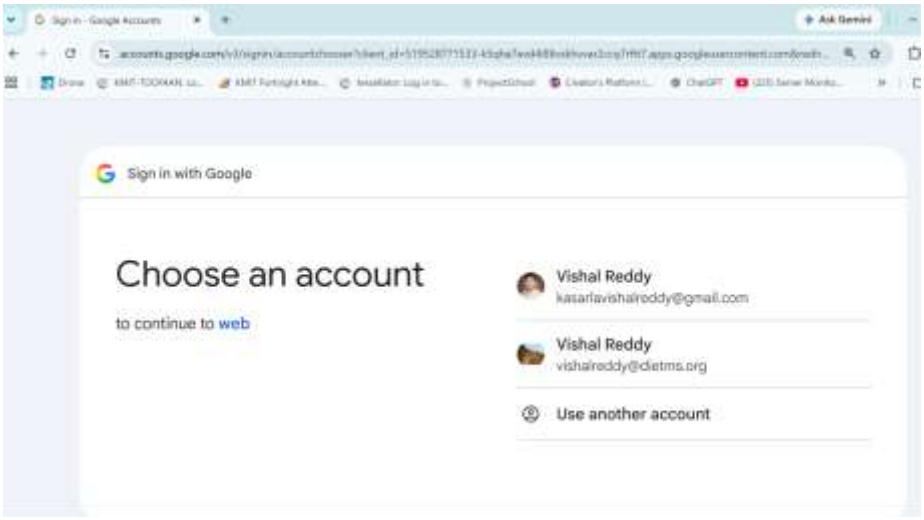
4. Start Google Login in the Browser

Do not begin by sending the Google login URL as a normal Thunder Client request. Google OAuth contains an interactive redirect/login/consent step that is designed for a browser.

Open the following URL in a browser:

```
http://localhost:3000/api/auth/google
```





The route normally looks like this:

```
router.get(
  "/google",
  passport.authenticate(
    "google",
    {
      scope: ["profile", "email"]
    }
  )
);
```

The browser is redirected to Google, where the user selects an account or signs in and grants the requested profile/email permissions.

5. Google Redirects to the Callback

After successful Google authentication, Google redirects the browser to:

```
http://localhost:3000/api/auth/google/callback
```

Your callback route authenticates the Google response and then calls the application controller:

```
router.get(
  "/google/callback",
  passport.authenticate(
    "google",
    {
      session: false,
      failureRedirect: "/api/auth/google/failure"
    }
  ),
  googleCallback
);
```

The controller then creates your application JWT and places it in an HttpOnly cookie.

```
const token = generateToken(user);

res.cookie(
  "accessToken",
  token,
  {
    httpOnly: true,
    secure: false,
    sameSite: "lax",
    maxAge: 15 * 60 * 1000
  }
);
```

6. Verify the Google User in MongoDB

After the first successful Google login, verify that the user was created or linked in the users collection.

```
{
  "_id": "...",
  "name": "Ravi",
  "email": "ravi@gmail.com",
  "googleId": "1083...",
  "provider": "google",
  "role": "user",
  "active": true
}
```

A Google-created account does not need a local password unless you explicitly support both login methods for the same account.

7. Why the Browser Cookie Is Not Automatically Available in Thunder Client

The accessToken cookie created after the OAuth callback belongs to the browser that completed the login. Thunder Client maintains a separate cookie store, so it will not automatically receive the browser cookie.

```
Browser OAuth Cookie
        ≠
Thunder Client Cookie Store
```

Therefore, calling the following immediately from Thunder Client may return 401:

```
GET http://localhost:3000/api/auth/me
```

```
{
  "message": "Authentication required"
}
```

8. Development Method: Temporarily Return the JWT

For classroom/testing purposes, temporarily include the generated token in the JSON response from googleCallback(). This makes it easy to copy the JWT into Thunder Client.

```
const googleCallback = async (req, res) => {
  try {
    const user = req.user;

    if (!user) {
      return res.status(401).json({
        message: "Google authentication failed"
      });
    }

    const token = generateToken(user);

    res.cookie("accessToken", token, {
      httpOnly: true,
      secure: false,
      sameSite: "lax",
      maxAge: 15 * 60 * 1000
    });

    res.status(200).json({
      message: "Google login successful",
      token,
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role
      }
    });

  } catch (error) {
    res.status(500).json({
      message: error.message
    });
  }};
```

> **Development only:** Remove token from the JSON response in the final version. Keep the production token only in the HttpOnly cookie.

9. Method A - Use the JWT as a Cookie in Thunder Client

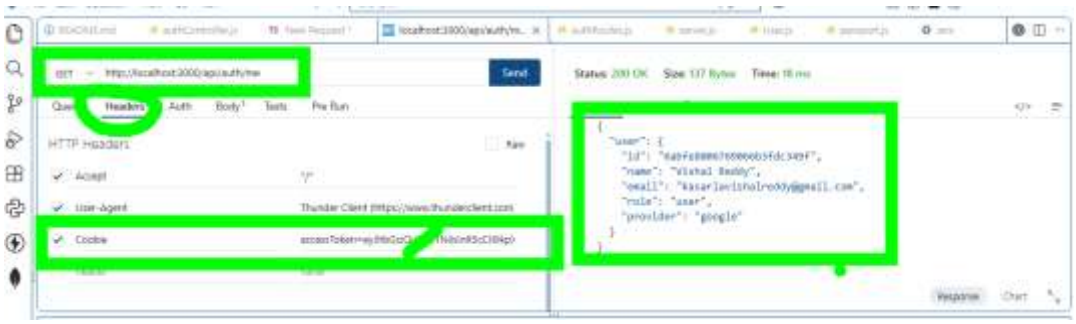
Copy the JWT returned by the temporary callback response. In Thunder Client, create the request:

```
GET http://localhost:3000/api/auth/me
```


```
{"message":"Google login
successful","token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.e
yJ1c2VySWQiOiI2YWJmZTg4MDY3NjkwNjZiM2ZkYzM0OWYiLCJpYXQiOjE3
OTA5NjI4MjIsImV4cCI6MTc5MDk2MzcyMn0.LWaBxilD4guArHMYyi0SQJn
yeLQUOc_M_9KLoT3alik","user":
{"id":"6abfe88067690066b3fdc349f","name":"Vishal
Reddy","email":"kasarlavishalreddy@gmail.com","role":"user"
,"provider":"google"}}
```


Add this request header:

```
Cookie: accessToken=eyJhbGciOiJIUzI1NiIs...
```



If authentication succeeds, the response should contain the current user information.

```
{
  "user": {
    "id": "...",
    "name": "Ravi",
    "email": "ravi@gmail.com",
    "role": "user"
  }
}
```

10. Recommended Method - Support Bearer Tokens for API Testing

For Thunder Client, it is more convenient to let jwtAuth.js accept either the HttpOnly cookie or an Authorization: Bearer header. The browser can continue using the cookie while API tools can use Bearer authentication.

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const requireJwtAuth = async (req, res, next) => {
  try {
    let token = req.cookies.accessToken;

    const authHeader = req.headers.authorization;

    if (
      !token &&
      authHeader &&
      authHeader.startsWith("Bearer ")
    ) {
      token = authHeader.split(" ")[1];
    }

    if (!token) {
      return res.status(401).json({
        message: "Authentication required"
      });
    }

    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );

    const user = await User
      .findById(decoded.userId)
      .select("-password");

    if (!user) {
      return res.status(401).json({
        message: "User not found"
      });
    }

    req.user = user;
    next();

  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token"
    });
  }
};

module.exports = requireJwtAuth;
```

11. Thunder Client Steps Using Bearer Authentication

1. Open Thunder Client in VS Code.
2. Create a new request.
3. Set the method and protected endpoint, for example GET http://localhost:3000/api/auth/me.
4. Open the Auth tab.
5. Choose Bearer Token.
6. Paste the JWT copied from the Google OAuth callback response.
7. Send the request.
8. Repeat the same authentication method for other protected Student routes.

12. Test Role-Based Access Control (RBAC)

New Google users should normally be created with role = user. Use the same JWT to test an admin-only Student endpoint.

```
GET http://localhost:3000/api/students
```

If the route uses allowRoles("admin"), a normal Google user should receive:

```
HTTP 403
{
  "message": "Access denied"
}
```

For an admin test, update the trusted user in MongoDB to role = admin, authenticate again, and obtain a new JWT. Then repeat the protected request.

```
Google
  ↓
Authentication

MongoDB role
  ↓
Authorization / RBAC
```

13. Complete End-to-End Testing Flow

```
1. Start Node/Express backend
        ↓
2. Open /api/auth/google in browser
        ↓
3. Complete Google login
        ↓
4. Google calls /api/auth/google/callback
        ↓
5. Backend finds/creates User in MongoDB
        ↓
6. Backend generates application JWT
        ↓
7. Copy JWT during development
        ↓
8. Thunder Client → Bearer Token
        ↓
9. Test /api/auth/me
        ↓
10. Test Student CRUD + RBAC
```